

## 10 آشنایی با DOT MATRIX و LCD گرافیکی

### Dotmatdix 10.1

ماتریس نقطه‌ای 8x8 یک نمایشگر LED است که شامل 64 LED (8 ردیف و 8 ستون) می‌باشد. این نوع نمایشگر به دلیل اندازه کوچک و قابلیت‌های بصری جذاب، در پروژه‌های مختلف الکترونیکی محبوب است. در ادامه توضیحاتی در مورد کاربردها و نحوه استفاده از آن آورده شده است:

نمایش متن و اعداد: می‌توان از ماتریس برای نمایش حروف، اعداد و کاراکترهای خاص استفاده کرد.

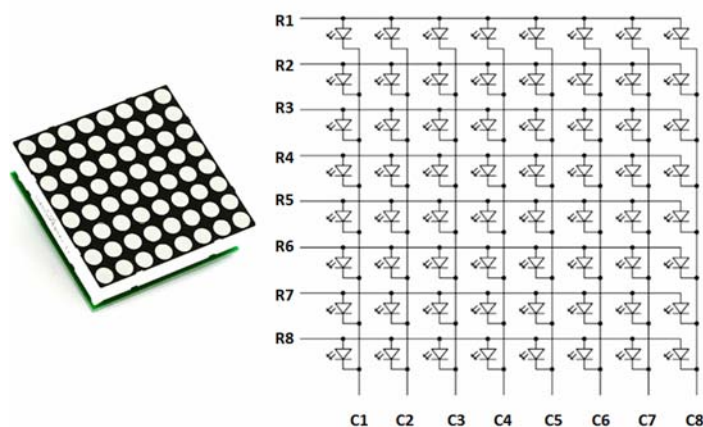
انیمیشن‌ها: با تغییر سریع حالت LEDها، می‌توان انیمیشن‌های ساده ایجاد کرد.

بازی‌های ساده: طراحی بازی‌های ساده مانند "موش و گربه" با استفاده از ماتریس.

نمایش علائم و نشانه‌ها: نمایش علائم هشدار یا نشانه‌های خاص.

ساعت دیجیتال: نمایش زمان به صورت دیجیتال.

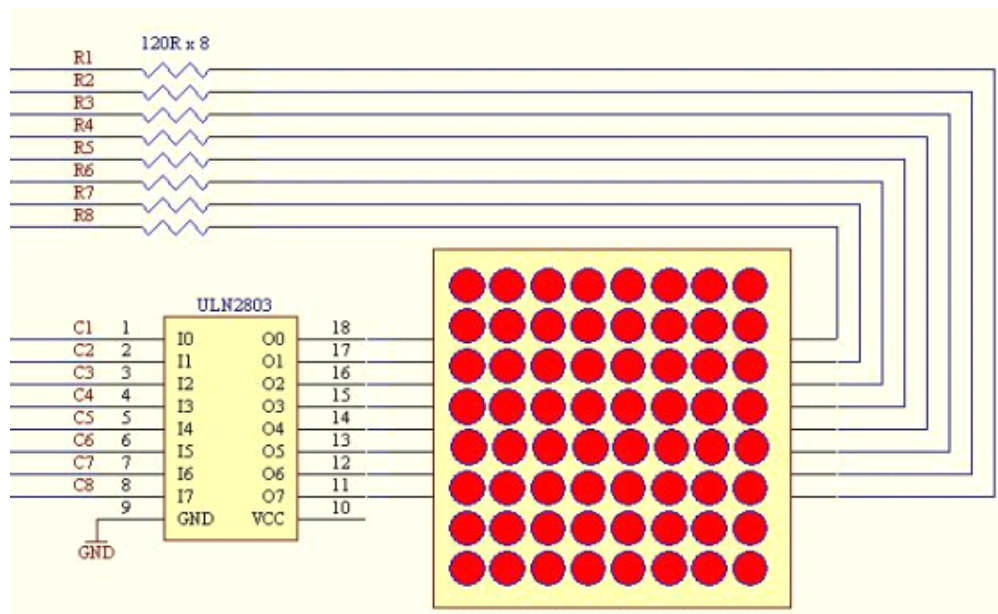
این نمایشگرها مانند شکل 1-10 از تعدادی LED تشکیل شده‌اند که در هر ردیف دارای آندهای مشترک و در هر ستون با کاتدهای مشترک می‌باشد. برای روشن شدن هر یک از LEDها بایستی سطر و ستون مورد نظر به نحو مناسب فعال گردد.



شکل 1-10: نمایی از ساختار Dot-Matrix

در مجموعه آموزشی آزمایشگاه، یک عدد نمایشگر Dot-Matrix از نوع 8x8 (8 ستون و 8 ردیف) به رنگ قرمز در بلوکی تحت عنوان DotMatrix Display قرار داده شده است. ستون‌ها توسط C1 تا C8 و ردیف‌ها توسط حروف R1 تا R8 نام‌گذاری شده‌اند. لذا برای روشن نمودن هر کدام از LEDها می‌توان به ستون مربوط به همان LED صفر منطقی و به ردیف متناظر یک منطقی اعمال نمود. شماتیک مربوط به این بلوک در شکل 2-10

مشاهده می‌شود. در این مدار برای تامین جریان led ها از uln2803 استفاده شده است که خروجی های آن معکوس ورودی هستند. لذا در برنامه نویسی با میکروکنترلر سطر ها و ستون ها با یک فعال خواهند شد.

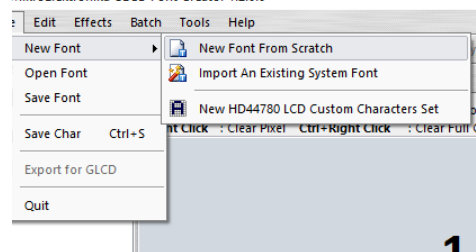


شکل 10-2 - ساختار نمایشگر DotMatrix تعبیه شده بر روی پکیج آزمایشگاه

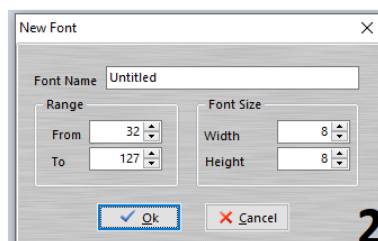
برای نمایش حروف و علائم روی 64 عدد LED، از روش جاروب کردن سطر و ستون استفاده می‌گردد، به این صورت که ابتدا سطر اول برابر با یک و مابقی سطرها صفر می‌گردد. سپس مقادیر ستون‌ها با توجه به وضعیت خاموش یا روشن بودن هر LED در این سطر تنظیم می‌شود. بعد از تاخیر زمانی مشخص، سطر دوم برابر با یک و مابقی سطرها صفر شده و مقادیر مربوط به ردیف دوم بر روی 8 پایه متناظر با ستون‌ها قرار می‌گیرد. این روند تا سطر هشتم ادامه می‌یابد. (عمل جاروب کردن را می‌توان با شیفت دادن صفر منطقی روی ستون‌ها و قرار دادن مقادیر روی ردیف‌ها نیز انجام داد). برای این که طرح مورد نظر بر روی Dot-Matrix روشن بماند، پروسه ذکر شده با تاخیرهای در حد چند میلی ثانیه به دفعات اجرا می‌گردد و با توجه به خطای دید در چشم انسان، طرح مورد نظر بر روی کل 64 عدد LED در هر لحظه مشاهده می‌گردد. مشابه این روش در LCDهای کاراکتری و برای ایجاد کاراکترهای خاص (مانند علائم و حروف فارسی) نیز استفاده می‌گردد.

### 10.1.1 ایجاد قلم برای نمایش متون روی Dot-Matrix

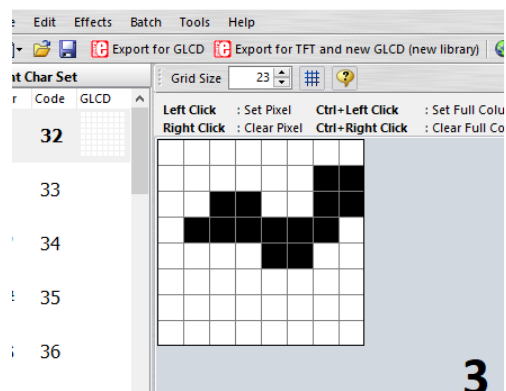
برای ایجاد قلم جهت نمایش نشانه‌های نوشتاری مختلف می‌توان از نرم‌افزارهای جانبی مانند GLCD Font Creator استفاده نمود. مراحل مختلف کار با این نرم‌افزار در شکل 10-3 نشان داده شده است.



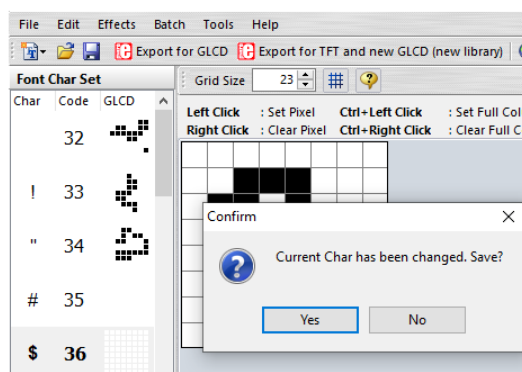
1



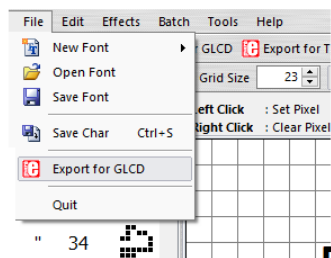
2



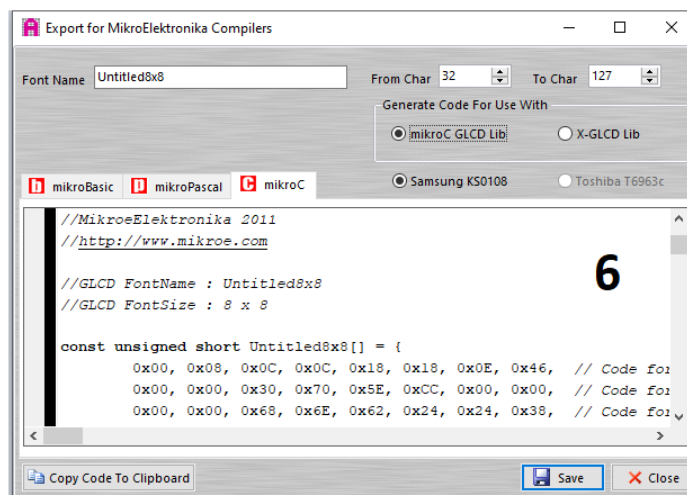
3



4



5



6

- 1: قلم جدید ایجاد می گردد .
- 2: می توان تعداد کاراکترها را محدود کرد.
- 3: می توان قلم مورد نظر را طراحی نمود.
- 4: با کلیک روی آرایه بعدی (مانند کد 33)، پنجره ذخیره نشان داده می شود.
- 5: کدهای متناظر با قلمها ایجاد می گردد.
- 6: می توان تمام کدها را انتخاب و پس از copy کردن در یک فایل txt ذخیره نمود و یا از save استفاده کرد. (در برخی از موارد به سبب اشکالات برنامه، گزینه Save فایلی را ذخیره نمی کند).

شکل 10-3: مراحل کار با GLCD Font Creator

## 10.1.2 اصول کلی نمایش متن بر روی DotMatrix

داده‌های خروجی حاصل از نرم‌افزار ایجاد قلم برای هر نشانه را به عنوان مقادیر مربوط به ستون‌های Dot-Matrix مطابق برنامه 1-10 و مقادیر مربوط به روند جاروب کردن سطرها را مطابق برنامه 2-10، در ساختارهای آرایه‌ای تعریف و ذخیره می‌نماییم.

```
برنامه 1-10 char alphabet[][8]={
{ 0x06, 0x29, 0x29, 0x29, 0x1F, 0x00, 0x00, 0x00},// a Locate in array = 0
{ 0xFF, 0x09, 0x11, 0x11, 0x0E, 0x00, 0x00, 0x00},// b Locate in array = 1
{ 0x1E, 0x21, 0x21, 0x21, 0x12, 0x00, 0x00, 0x00},// c Locate in array = 2
{ 0x7F, 0x88, 0x88, 0x88, 0x7F, 0x00, 0x00, 0x00},// A Locate in array = 3
{ 0xFF, 0x91, 0x91, 0x91, 0x6E, 0x00, 0x00, 0x00},// B Locate in array = 4
{ 0x7E, 0x81, 0x81, 0x81, 0x42, 0x00, 0x00, 0x00},// C Locate in array = 5
};
```

```
برنامه 2-10 unsigned char R_data[8] = {0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80};
```

در ادامه یک ساختار switch/case مطابق برنامه 3-10 پیاده‌سازی می‌گردد که در آن به ازای هر نشانه، یک case مجزا در نظر گرفته می‌شود. با اجرای این ساختار، موقعیت داده متناظر با حرف مورد نظر در متغیر locat ذخیره می‌شود. به عنوان مثال با در نظر گرفتن برنامه 1-10، برای حرف اول (a) مقدار صفر را برمی‌گرداند، برای حرف دوم (b) مقدار 1 و همین طور برای مقادیر دیگر موقعیت را برمی‌گرداند.

```
برنامه 3-10 void lookup(char input)
{
switch (input)
{
case 'a' :
locat=0;
break;
case 'b' :
locat=1;
break;
case 'c' :
locat=2;
break;
case 'A' :
locat=3;
break;
case 'B' :
locat=4;
break;
case 'C' :
locat=5;
break;
}
}
```

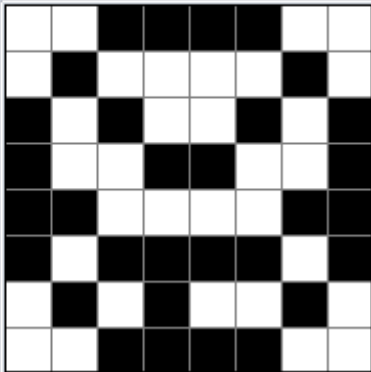
}

اگر کارکتر ذیل معرفی شده باشد با استفاده از کد زیر میتوان کارکتر را روی dotmatrix نشان داد.

| Char | Code | GLCD                                                                              |
|------|------|-----------------------------------------------------------------------------------|
|      | 32   |  |
| !    | 33   |                                                                                   |
| "    | 34   |                                                                                   |
| #    | 35   |                                                                                   |

Left Click : Set Pixel    Ctrl+Left Click

Right Click : Clear Pixel    Ctrl+Right Click



```
#include "main.h"

char row[8]={0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80};
char cul[8]={0x3C, 0x52, 0xA5, 0xE9, 0xA9, 0xA5, 0x52, 0x3C};

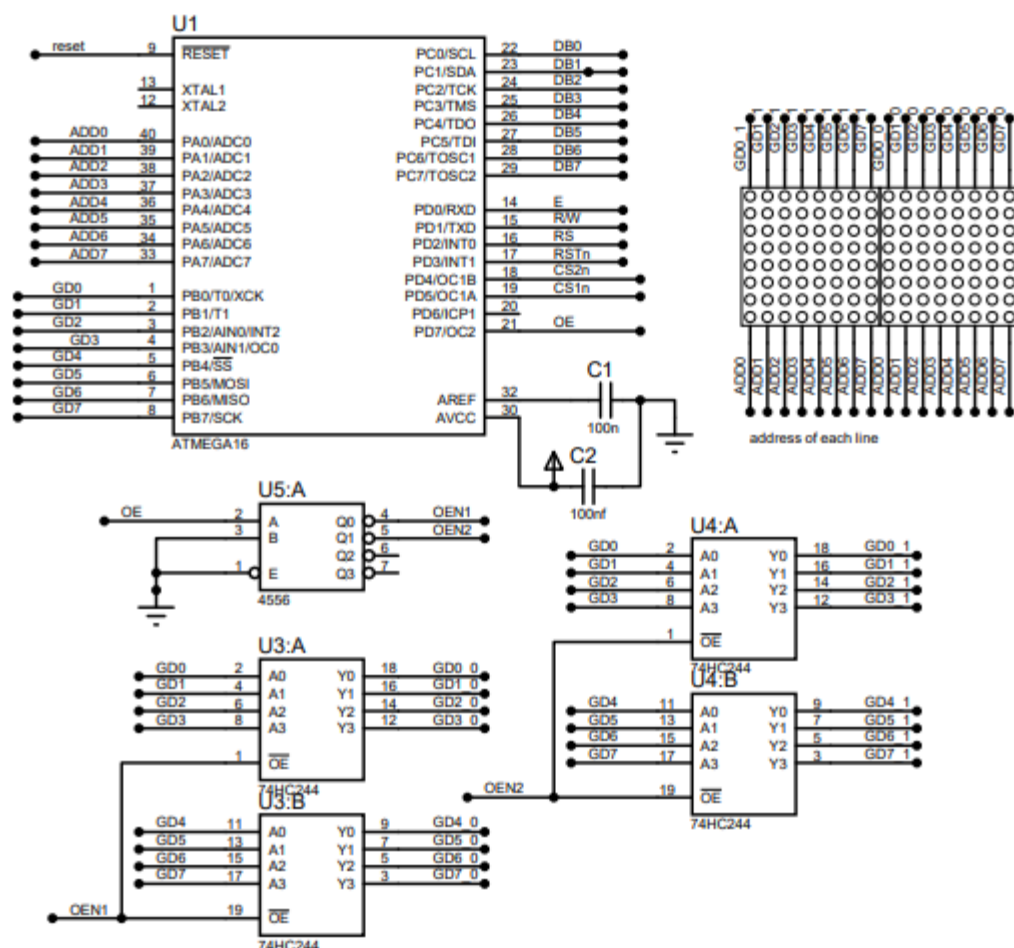
void SystemClock_Config(void);
static void MX_GPIO_Init(void);

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    while (1)
    {
        for(int i=0;i<8;i++){
            GPIOE->ODR =cul[i];
            GPIOA->ODR =row[i];
            HAL_Delay(1);
        }
    }
}
```

### 10.1.3 استفاده از چند ماژول DotMatrix

با توجه به این که هر ماژول به دو درگاه جهت سامان‌دهی ستون‌ها و ردیف‌ها نیاز دارد، در صورت استفاده بیش از یک ماژول، می‌توان سطرهای همه ماژول‌ها را به یکی از درگاه‌ها متصل کرد و برای ردیف‌ها از تعدادی D-Flip Flop استفاده نمود. نمایی از این سخت‌افزار در شکل 10-4 مبتنی بر میکروکنترلر ATMEGA16 نشان داده شده است. در این سخت‌افزار از درگاه A برای سطرها و از درگاه B به صورت مشترک برای ستون‌ها استفاده شده است. در این سخت‌افزار با فعال نمودن پایه OE، داده‌های ارسالی از سوی ریزپردازنده در حافظه خارجی ثبت می‌شود و با غیرفعال شدن این پایه، این مقادیر ثابت باقی خواهند ماند. بدین ترتیب در بازه‌های زمانی مناسب می‌توان داده‌های هر ستون ماژول DotMatrix را به صورت مجزا مقدار دهی کرد. برای مدیریت سیگنال‌های OE، می‌توان از یک انکدر استفاده نمود تا بتوان با استفاده از تعداد پایه‌های کمتری از ریزپردازنده، سیگنال‌های OE بیشتری را کنترل نمود.



شکل 10-4: راه‌اندازی چند ماژول DotMatrix به صورت همزمان

## 10.2 LCD گرافیکی

LCD گرافیکی 64x128 یکی از نمایشگرهای محبوب است که برای نمایش متون و گرافیک‌های ساده استفاده می‌شود. این نوع نمایشگر معمولاً با پروتکل‌های SPI یا I2C و یا پارالل به میکروکنترلرها متصل می‌شود. در اینجا نحوه اتصال و راه‌اندازی یک LCD گرافیکی 64x128 به STM32F407 را بررسی می‌کنیم.

اساس کار GLCD<sup>1</sup> مشابه DotMatrix است ولی واحدهای نمایشگر نورانی در آن بسیار کوچکتر می‌باشد. بنابراین برای نمایش داده‌ها روی آن نیز از روش جاروب کردن سطر استفاده می‌شود، بدین صورت که در هر مرحله تنها یکی از سطرها یک و مابقی صفر می‌شوند و در این لحظه، داده‌های مربوط به آن سطر نوشته می‌شوند. با توجه به تراکم بالای نقاط نورانی در GLCD، برای تعیین زمان‌بندی روشن و خاموش شدن هر سطر و ستون نیاز به یک راه‌انداز<sup>2</sup> می‌باشد.

برای راه‌اندازی ستون‌های GLCD، از تراشه‌ی NT7108C استفاده می‌شود. این تراشه، شامل یک RAM، یک لچ 64 بیتی و یک دیکدر می‌باشد. حافظه RAM برای ذخیره داده‌هایی که از طریق ریزپردازنده منتقل می‌شود، به کار می‌رود.

تراشه NT7107C، نیز یک راه‌انداز 64 کاناله است که در فاصله‌های زمانی مشخص، هر یک از سطرهاى GLCD را فعال می‌نماید تا داده‌های ذخیره شده درون NT7108C بر روی GLCD نمایش داده شود. در حقیقت، این تراشه عملیات جاروب کردن سطرها را انجام می‌دهد.

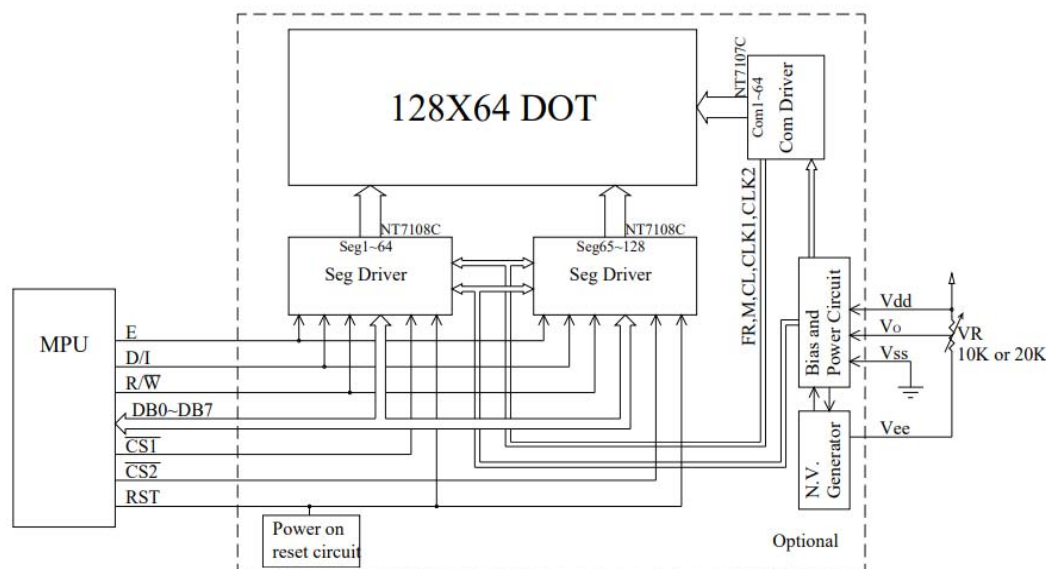
GLCD مورد استفاده در برد آزمایشگاه دارای ابعاد 128\*64 پیکسل می‌باشد و به همین دلیل مطابق شکل 5-10 از دو تراشه NT7108C و یک تراشه‌ی NT7107C برای راه‌اندازی نمودن 128 ستون و 64 سطر آن استفاده می‌شود. این تراشه‌ها با تراشه‌های KS0108B و KS0107B از شرکت سامسونگ سازگار هستند و می‌توان در تنظیمات CodeVision آن‌ها را انتخاب نمود.

---

<sup>1</sup> Graphical LCD

<sup>2</sup> Drive

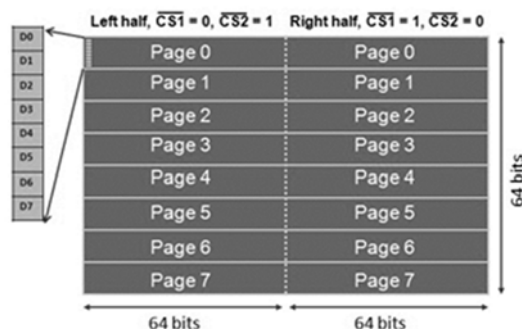




شکل 10-5: مدار راه انداز GLCD در پکیج آزمایشگاهی

تراشه‌ی NT7107C، 64 سطرنمایشگر را (COM1 – COM64) را راه اندازی می‌کند. در هر لحظه از زمان، تنها یکی از خطوط COM فعال می‌باشد. اولین تراشه NT7108C (IC1) سگمنت‌های نیمه سمت چپ را فعال می‌کند و دومین تراشه NT7108C (IC2)، سگمنت‌های نیمه سمت راست را فعال می‌نماید. با استفاده از پین‌های CS13 و CS24 می‌توان به طور جداگانه به هر یک از نیمه‌های نمایشگر دست یافت. هر کدام از نیمه‌ها مانند شکل 10-6 شامل 8 صفحه افقی با شماره‌های 0-7 می‌باشد که هر کدام از 64 بایت تشکیل شده است.

مقداردهی از صفحه صفر شروع می‌شود. اگر یک بایت داده به GLCD انتقال داده شود، در اولین ستون صفحه صفر نمایش داده می‌شود (شکل 10-6). اگر این کار برای 63 بایت داده تکرار شود و سپس با فعال‌سازی نیمه دوم، برای 64 داده دیگر ادامه یابد، در واقع 8x128 پیکسل از GLCD که همگی در صفحه صفر قرار دارند، مقداردهی خواهند شد. این کار برای سایر صفحات نیز به همین صورت تکرار می‌گردد.



<sup>3</sup> Chip Select 1

<sup>4</sup> Chip Select 2



## 10.2.1 آشنایی با پایه‌های GLCD

پایه‌های GLCD مانند جدول 1-10 به دو دسته تقسیم می‌شوند. دسته نخست پایه‌های داده (DB0-DB7) که وظیفه انتقال داده را برعهده دارند و دسته‌ی دیگر پایه‌های کنترلی هستند. پایه‌های کنترلی به شرح زیر می‌باشند:

R/W: زمانی که این پایه مقدار یک داشته باشد، داده‌های موجود در DB[7:0] را نمایش می‌دهد و زمانی که  $E=1$ ، CS1B=0 و CS2B=0 باشد، مقدار آن توسط پردازنده می‌تواند خوانده شود.

CS1 و CS2: زمانی که CS1=1 و CS2=0 باشد، تراشه اول NT7108C (IC1) انتخاب می‌شود و زمانی که CS1=0 و CS2=1 باشد تراشه دوم NT7108C (IC2) انتخاب می‌شود.

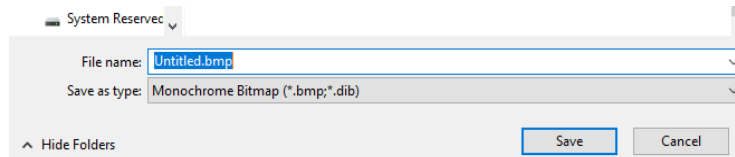
RS: اگر RS=0 باشد، GLCD دستورالعمل‌های دریافتی را اجرا می‌کند، در غیر این صورت مقدار خوانده شده از پایه‌های داده را در Data Register قرار می‌دهد.

جدول 1-10: مشخصات پایه‌های GLCD

|    | Name            | Level    | Function                                     |
|----|-----------------|----------|----------------------------------------------|
| 1  | CS1             | L        | Chip Select 1 (segment 1-64)                 |
| 2  | CS2             | L        | Chip Select 2 (segment 65-128)               |
| 3  | Vss             | 0V       | Ground                                       |
| 4  | Vcc             | 5.0V     | 5v Supply in+                                |
| 5  | V0              | variable | Contrast Adjust                              |
| 6  | RS or D/I or CD | H/L      | H: Data , L: Instruction                     |
| 7  | R/W             | H/L      | H: Read(MPU← Module) , L :Write(MPU→ Module) |
| 8  | E               | H        | ENABLE SIGNAL                                |
| 9  | DB0             | H/L      | Data bus line                                |
| 10 | DB1             | H/L      | Data bus line                                |
| 11 | DB2             | H/L      | Data bus line                                |
| 12 | DB3             | H/L      | Data bus line                                |
| 13 | DB4             | H/L      | Data bus line                                |
| 14 | DB5             | H/L      | Data bus line                                |
| 15 | DB6             | H/L      | Data bus line                                |
| 16 | DB7             | H/L      | Data bus line                                |
| 17 | RST             | L        | Reset the LCM                                |
| 18 | VEE             |          | Negative Voltage Output                      |
| 19 | A               |          | Power supply for B/L(+)LED BACKLIGHT         |
| 20 | K               |          | Power supply for B/L(-)LED BACKLIGHT         |

## 10.2.2 نمایش تصویر روی GLCD

با توجه به این که GLCD تعبیه شده در پکیج آزمایشگاه، سیاه و سفید است، صرفاً تصاویر محدودی را می‌توان بر روی آن نشان داد. این تصاویر باید دارای قالب monochrome باشند که می‌توان عکس مورد نظر را در محیط paint و مطابق شکل 7-10 در این قالب ذخیره نمود. همچنین برای جلوگیری از افت کیفیت بهتر است اندازه عکس 12864x پیکسل باشد.

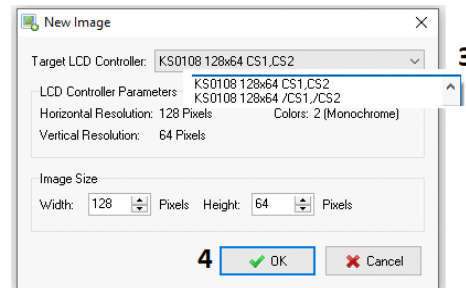


شکل 7-10: ذخیره تصویر در محیط Paint

## 10.2.3 تهیه کدهای عکس با استفاده از نرم‌افزار LCD Vision

برای ذخیره تصاویر در حافظه پردازنده و به منظور نمایش بر روی GLCD باید آن‌ها را به صورت آرایه دو بعدی از داده‌های باینری در آورد. البته جهت فشرده‌سازی از کد مبنای 16 به جای باینری استفاده می‌شود. به این منظور پس از آماده‌سازی اولیه عکس مورد نظر، می‌توان در محیط LCD Vision کدهای مبنای 16 تصاویر را تهیه نمود که مراحل آن در شکل 8-10 قابل مشاهده است.

- 1- نرم افزار LCD Vision را باز نمایید.
- 2- از منوی file گزینه new Image را بزنید.
- 3- در پنجره باز شده کنترلر مربوطه را انتخاب نمایید.
- 4- گزینه ok را بزنید.



- 5- در پنجره باز شده، تصویر را با استفاده از موس رسم نمایید یا از منوی file گزینه Import File انتخاب نمایید.
- 6- در صورتی که Import File انتخاب شده باشد، در پنجره باز شده تصویر مورد نظر را انتخاب و open نمایید.
- 7- در پنجره باز شده گزینه ok را انتخاب نمایید.



- 8- بخشی از تصویر که قصد دارید روی GLCD نمایش دهید را با موس انتخاب نمایید.
- 9- از منوی File گزینه Export را انتخاب نمایید.
- 10- در پنجره باز شده گزینه ok را انتخاب نمایید.
- 11- کدهای تولید شده، در پنجره ی Export Preview نشان داده می‌شود.
- 12- از منوی file گزینه Save Export را انتخاب نمایید.
- 13- در پنجره باز شده گزینه ok را انتخاب نمایید.
- 14- مسیر مورد نظر را انتخاب و ذخیره نمایید.
- 15- در پنجره باز شده اعلام می نماید که دو عدد فایل با فرمت h و c ایجاد شده است.



شکل 8-10: مراحل تهیه کدهای مبنای 16 تصویر در محیط LCD Vision

فایل با فرمت C حاوی کدهای موردنظر برای نمایش عکس است. این فایل به‌طور مستقیم برای فراخوانی در CodeVision و استفاده از کتابخانه LCD گرافیکی مناسب است. اما برای استفاده در Keil و سازگاری با کدهای نوشته‌شده، باید بخشی از آن که حاوی داده های بخش عمودی است را انتخاب کنید و در فایل image.h در Keil ذخیره نمایید.

```

17 flash unsigned char new_image[]=
18 {
19     /* Image width: 128 pixels */
20     0x80, 0x00,
21     /* Image height: 64 pixels */
22     0x40, 0x00,
23     #ifndef _GLCD_DATA_BYTE_
24     /* Image data for monochrome displays organized
25      as horizontal rows of bytes */
26     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
27     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
28     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,

```



این بخش ها نشان داده نشده

```

152 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
153 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
154 #else
155 /* Image data for monochrome displays organized
156 as rows of vertical bytes */
157 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,

```



این بخش ها نشان داده نشده

```

282 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
283 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
284 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
285 #endif
286 };

```



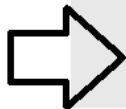
این بخش از داده را انتخاب  
نمایید

Image.h\*

```

1
2 #include "pgmspace.h"
3
4
5 const char img[1024] PROGMEM = {
6     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
7     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0xC0, 0xE0, 0xE0, 0xF0, 0xF8, 0xFC,
8     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,

```



داده ها را در این آرایه ذخیره کنید

## 10.2.4 نمایش تصویر روی LCD با استفاده از توابع HAL

برای نمایش تصویر روی LCD با استفاده از توابع HAL بایستی ابتدا تنظیمات مربوط به کلاک میکروکنترلر را در محیط stm32cube را انجام دهید. در این برنامه به صورت LL پایه های مورد نظر تنظیم شده و دیگر نیازی به تنظیم در محیط STM32cube نیست.

لذا در محیط Keil تغییرات ذیل را در برنامه لحاظ نمایید.

کپی کردن فایل های برنامه و هدر فایل های مربوطه شامل glcd.c و delay\_fun.c و Font\_Header.h و Image.h و pgmspace.h در مسیر پروژه شامل پوشه Src و Inc

اضافه کردن فایل glcd.c و delay\_fun.c به پروژه

فراخوانی هدر فایل های glcd.h و delay\_fun.h

استفاده از دستورات لازم معرفی شده در glcd.c

## 10.2.5 فایل GLCD.c

فایل GLCD.c شامل موارد ذیل می باشد.

### 10.2.5.1 فراخوانی کتابخانه ها :

- فراخوانی فایل های هدر برای دسترسی به توابع و تعاریف مربوط به پروژه، از جمله توابع ورودی و خروجی استاندارد و هدرهای خاص برای قلم و تصاویر.

برای کار با GLCD کتابخانه های بسیاری معرفی شده اند که به عنوان نمونه یکی از آنها را انتخاب مینماییم . این کتابخانه دارای زیربرنامه های ذیل برای اجرای فرامین مختلف GLCD است.

### 10.2.5.2 تعریف پایه ها :

- تعریف پایه های کنترلی (RS, RW, EN, CS1, CS2, RST) که برای ارتباط با GLCD استفاده می شوند. با توجه به ساختار فایل کتابخانه ای، تعریف پایه ها با استفاده از #define انجام می شود.

```
#define RS      8           //pins on GPIOE
#define RW      9
#define EN     10
#define CS1    12
#define CS2    13
#define RST    14
```

## 10.2.5.3 تنظیمات پورت :

- توابعی برای تنظیم جهت و وضعیت پایه‌های داده و کنترل (تنظیم به حالت بالا یا پایین) که برای ارسال دستورات و داده‌ها به GLCD استفاده می‌شوند.

```
void control_high(int pin)
{
    GPIOE->ODR |= (1<< pin);
}
```

---

```
void control_low(int pin)
{
    GPIOE->ODR &= ~(1<< pin);
}
```

---

```
void control_xor(int pin)
{
    GPIOE->ODR ^= (1<< pin);
}
```

---

```
void Data_Port(int data)
{
    GPIOE->ODR=(GPIOE->ODR &0xfffff00)|(data &0x000000ff);
}
```

---

```
void DataCom_Port_Dir(void)
{
    RCC->AHB1ENR |= 0x10; //(1<<4); GPIOE

    GPIOE->MODER = 0x55555555;
    GPIOE->OTYPER = 0x00000000 ;
    GPIOE->OSPEEDR =0xAAAAAAAA ;
    GPIOE->PUPDR =0x00000000 ;
}
```

#### 10.2.5.4 توابع GLCD

- GLCD\_Command : ارسال دستورات به GLCD.
- GLCD\_Data : ارسال داده‌ها به GLCD.
- GLCD\_Init : راه‌اندازی و پیکربندی اولیه GLCD.
- GLCD\_ClearAll : پاک‌سازی تمام محتویات نمایشگر.
- GLCD\_String : نمایش رشته‌ای از متن بر روی GLCD.
- GLCD\_Image : نمایش یک تصویر بر روی GLCD.
- GLCD\_Animate : اجرای انیمیشن با نمایش متناوب تصاویر.

```
void GLCD_Command(char Command)
```

```
{  
    Data_Port(Command);  
    control_low(RS);  
  
    control_low(RW);  
    control_high(EN);  
    Delay_us(5);  
    control_low(EN);  
    Delay_us(5);  
}
```

---

```
void GLCD_Data(char Data)
```

```
{  
    Data_Port(Data);  
    control_high(RS) ;  
    control_low(RW) ;  
    control_high(EN);  
    Delay_us(5);  
    control_low(EN);  
    Delay_us(5);  
}
```

---

```
void GLCD_Init()
```

```
{  
    DataCom_Port_Dir();  
  
    control_high(CS1);  
    control_high(CS2);  
    control_high(RST);  
  
    Delay_ms(20);  
    GLCD_Command(0x3E);  
    /* Display OFF */  
}
```



```

        GLCD_Command(0x40);
        GLCD_Command(0xB8);
        GLCD_Command(0xC0);
        GLCD_Command(0x3F);
    }
    /* Set Y address (column=0)/*
    /* Set x address (page=0)/*
    /* Set z address (start line=0)/*
    /* Display ON/*

```

```

void GLCD_ClearAll()
{
    int i,j;

    control_high(CS1);
    control_high(CS2);

    for(i = 0; i < TotalPage; i++)
    {
        GLCD_Command((0xB8) + i);
        for(j = 0; j < 64; j++)
        {
            GLCD_Data(0)
        }
    }
    GLCD_Command(0x40);
    GLCD_Command(0xB8);
}

```

---

```

void GLCD_String(char page_no, char *str)
{
    unsigned int i, column;
    unsigned int Page = ((0xB8) + page_no);
    unsigned int Y_address = 0;
    float Page_inc = 0.5;

    control_high(CS1);
    control_low(CS2);

    GLCD_Command(Page);
    for(i = 0; str[i] != 0; i++)
    {
        if (Y_address > (1024-(((page_no)*128)+FontWidth)))
            break;
        if (str[i]!=32)
        {
            for (column=1; column<=FontWidth; column++)
            {
                if ((Y_address+column)==(128*((int)(Page_inc+0.5))))
                {

```

```

        if (column == FontWidth)
            break;
        GLCD_Command(0x40);
        Y_address = Y_address + column;
        control_xor(CS1);
        control_xor(CS2);
        GLCD_Command(Page + Page_inc);
        Page_inc = Page_inc + 0.5;
    }
}
if (Y_address > (1024 - (((page_no) * 128) + FontWidth)))
    break;
if ((font[(((str[i] - 32) * FontWidth) + 4)] == 0 || str[i] == 32)
{
    for(column=0; column<FontWidth; column++)
    {
        GLCD_Data(font[str[i]-32][column]);
        if((Y_address+1)%64==0)
        {
            control_xor(CS1);
            control_xor(CS2);
            GLCD_Command((Page+Page_inc));
            Page_inc = Page_inc + 0.5;
        }
        Y_address++;
    }
}
else
{
    for(column=0; column<FontWidth; column++)
    {
        GLCD_Data(font[str[i]-32][column]);
        if((Y_address+1)%64==0)
        {
            control_xor(CS1);
            control_xor(CS2);
            GLCD_Command((Page+Page_inc));
            Page_inc = Page_inc + 0.5;
        }
        Y_address++;
    }
    GLCD_Data(0);
    Y_address++;
    if((Y_address)%64 == 0)
    {
        control_xor(CS1);
        control_xor(CS2);
        GLCD_Command((Page+Page_inc));
        Page_inc = Page_inc + 0.5;
    }
}
}
GLCD_Command(0x40);

```

```

}

void GLCD_Image(const char* image)
{
    int column,page,page_add=0xB8,k=0;
    float page_inc=0.5;
    char byte;

    control_high(CS1);
    control_low(CS2);

    for(page=0;page<16;page++)
    {
        for(column=0;column<64;column++)
        {
            byte = pgm_read_byte(&image[k+column]);
            GLCD_Data(byte);
        }
        control_xor(CS1);
        control_xor(CS2);
        GLCD_Command((page_add+page_inc));
        page_inc=page_inc+0.5;
        k=k+64;
    }
    GLCD_Command(0x40);
    GLCD_Command(0xB8);
}

```

---

```

void GLCD_Animate(void)
{
    GLCD_Init();

    GLCD_ClearAll();

    GLCD_String(0,"Atmel");

    while(1){

        GLCD_Image(img);
        Delay_ms(500);
        GLCD_Image(img1);
        Delay_ms(500);
    }
}

```

## 10.2.6 پروژه نمایش انیمیشن ساده

اصلاح برنامه مشابه کدهای ذیل

```
#include "main.h"

#include "glcd2.h"
#include "delay_fun.h"

extern const char img[1024];
extern const char img1[1024];

void SystemClock_Config(void);
static void MX_GPIO_Init(void);

int main(void)
{

    HAL_Init();

    SystemClock_Config();

    MX_GPIO_Init();

    TIM6Config ();
    GLCD_Init();
    GLCD_ClearAll();
    /* Initialize GLCD */
    /* Clear all GLCD display */

    GLCD_String(0,"Microlab STM32f407 1403");
    /* Print String on 0th page of display
    */
    HAL_Delay(2000);

    while (1)
    {

        GLCD_Image(img);
        HAL_Delay(500);
        GLCD_Image(img1);
        HAL_Delay(500);

    }

}
```